

Production Launch & Backend Integration Master Command

Sixteen commands, one per tab, from a live production incident to a fully integrated `/api/v1` client and a public release.

Field	Value
Prepared	19 August 2026 — every figure measured on this date, from GitHub <code>main</code> and from live requests to production
Backend	PaulEspinass2020/servana_api — GitHub <code>main</code> <code>44600a6</code> ; local diverged : 47 local-only / 17 remote-only commits
Client	Upupapp/ServanaClientAPP — GitHub <code>main</code> <code>80eff51</code> ; local <code>61374fa</code> , 24 unpushed, <code>1.0.0+40</code>
Production	<code>https://api.servana.com.ph</code> — probed live for this sweep
Scope	Backend GitHub- <code>main</code> sweep (278 suites, 5,957 tests), client local sweep (490 lib files, 110 test files, 1,502 tests), full client↔backend integration probe
Structure	16 tabs across 5 phases. Each tab is self-contained: objective, measured justification, scope, method, acceptance gate, guardrails, deliverable.

THE HEADLINE — PRODUCTION IS DOWN RIGHT NOW

Every catalog read returns **HTTP 500**, on both namespaces. `/api/v1/catalog` , `/api/v1/catalog/summary` , `/api/v1/catalog/categories` , `/api/catalog` , `/api/services` , `/api/services/full` — all 500. `/api/v1/search?q=aircon` also 500; it answers 200 only for an empty query, which short-circuits before the database.

These returned **200 at 2026-08-18T16:05Z**. `/healthz` is 200 and every gated route still answers 401, so the process is alive and authentication works. **What fails is every read that queries catalog or service data.**

A customer cannot browse services. A customer cannot book. This outranks every other item in this document.

THE SECOND FINDING — the repositories have diverged

The backend has **47 local-only** commits and **17 remote-only** commits. A plain `git push` is now rejected as non-fast-forward. Two agents and a human have been committing to one working tree, and no one has reconciled them. Five of those local commits are verified production fixes that have never shipped.

What was measured

Everything below derives from GitHub `main` of both repositories and from live requests to production on 19 August 2026. Nothing is carried forward from an earlier sweep without re-measurement — an earlier sweep is exactly what was wrong last time.

PRODUCTION, LIVE

Probe	Result	Reading
<code>/api/v1/catalog</code>	500	Was 200 yesterday. Envelope is canonical <code>{error: {code:"INTERNAL", requestId}}</code> , so the v1 error path itself works.
<code>/api/catalog</code> , <code>/api/services</code> , <code>/api/services/full</code>	500	The legacy surface the shipped app uses is down too. This is not a v1-only fault.
<code>/api/v1/search?q=aircon</code>	500	Empty query returns 200; a real query fails. Search short-circuits before the database.
<code>/healthz</code>	200	Process alive.
<code>/api/v1/me</code> , <code>/api/v1/bookings</code> , <code>/api/quote</code>	401	Auth middleware and DB-backed token checks work.
v1 401 envelope	legacy shape	<code>{"status":"failed"}</code> , no <code>requestId</code> . Fix committed locally, never deployed.
Link <code>rel="successor-version"</code> on <code>/api/catalog</code>	wrong	Points at <code>/api/v1/bookings/:bookingId</code> . Fix committed locally, never deployed.
HSTS, nosniff, frame-deny, referrer-policy	absent	All four. Implementation committed locally, never deployed.

The diagnostic pattern. Auth works, health works, the error envelope works — only queries against catalog and service data fail. That is a **schema** signature, not an application-code one. The two catalog commits deployed today touched `providerCatalogService.ts` only, which is the provider write path, not the customer read path. The same deploy carried `ledger-repair.sql`, which rewrites 38 rows of `servana.schema_migrations`, and the ledger includes `020-catalog-v2-expand.sql` — a catalog schema expand.

REPOSITORY STATE

Measurement	Backend	Client	Reading
GitHub <code>main</code>	44600a6	80eff51	What is deployed / released from.
Local HEAD	689e7b9	61374fa	—
Local-only commits	47	24	Backend includes 5 verified production fixes.
Remote-only commits	17	0	Backend has diverged ; the client is a clean fast-forward.
Suite on GitHub <code>main</code>	278 suites / 5,957 tests, exit 0	—	Green — the outage is not a test-visible regression.
Suite locally	295 suites / 6,223 tests, exit 0	1,502 passed / 6 skipped	Both green.
Client analyzer	—	0 issues	Was 42 infos; triaged to zero.
Push status	blocked — PAT lacks <code>workflow</code> scope		Both pushes carry workflow-file changes.

CLIENT INTEGRATION SURFACE

Measurement	Result	Reading
-------------	--------	---------

Implemented v1 contract paths mounted	105 / 105	Zero 404. v1 is fully deployed.
Legacy routes the shipped client constructs	70 / 70 answer	But six of them are among today's 500s.
Client files referencing <code>api/v1</code>	0	The canonical layer is not in this repository. The shipped app is 100% legacy.
Client legacy routes undeclared by the contract	30 / 70	No disposition, no successor, invisible to the deprecation clock.
Unreachable client screens	6 (1,417 lines)	Includes a checkout branch whose Continue button calls two unregistered routes.
Client DI graph	73 reg / 49 resolved	Intact — zero missing registrations.

THE SIXTEEN TABS

Tab	Command	Owner	Risk
Phase A — Stop the bleeding			
01	Restore the catalog: diagnose and fix the production 500	Backend	P0
02	Reconcile the diverged backend repository	Backend	High
03	Ship the three verified fixes that have never deployed	Backend	Medium
Phase B — Make the pipeline trustworthy			
04	A deploy that cannot silently break the catalog again	Backend	High
05	Migration safety: expand–migrate–contract, enforced	Backend	High
06	Unblock the push path and end the shared-tree collisions	All	Medium
Phase C — Integrate the client			
07	Land the canonical v1 client layer	Client	Blocker
08	The migration harness: evidence gate, rollout, rollback	Client + Backend	High
09	Wave 1–2 — catalog, search, identity and profile	Client	Medium
10	Wave 3–6 — the state-changing and money surfaces	Client	High
11	Close the 30 undeclared legacy routes	Backend	Medium
Phase D — Release engineering			
12	Finish the Android and iOS release paths	Client	High
13	Deep links: host the association files and verify on device	Client + Backend	Medium
14	Publish and rehearse the version gate	Client	High
Phase E — Quality and launch			
15	Front-end integrity: coverage matrix and dead code	Client	Medium
16	Launch readiness review and go / no-go	All	High

SEQUENCING. TAB 01 blocks everything — a launch programme is meaningless while the product is down. TABs 02 and 03 must precede any further deploy, because pushing a diverged tree is how the next outage happens. Phase C is gated on TAB 07: nine tabs of integration work cannot start while the layer they operate on is absent. Phases D and E run in parallel with C.

STANDING RULES. Measure, do not recall — every figure here was true on 19 August 2026 and some will not be when a tab is run. **Additive only** — one backend serves five clients; add fields, never rename or remove, and prove it by

diffing captured responses. **A green gate proves nothing until it has been watched to fail** — every detector in this document carries a mutation step. **Report what is not done as plainly as what is.**

TAB 01 — Restore the catalog: diagnose and fix the production 500

Phase	Owner	Risk	Blocks
Stop the bleeding	Backend	P0 — live outage	every other tab

OBJECTIVE

Return every catalog and service read to 200, establish what broke it, and leave behind a detector that would have caught it within a minute rather than a day.

WHY THIS, AND WHY NOW — MEASURED

- Measured 19 August 2026: `/api/v1/catalog`, `/api/v1/catalog/summary`, `/api/v1/catalog/categories`, `/api/catalog`, `/api/catalog/summary`, `/api/services` and `/api/services/full` all return **500**. The same paths returned **200 on 18 August at 16:05Z**.
- `/api/v1/search` returns 200 for an empty query and **500** for `?q=aircon`. The empty case short-circuits before touching the database, so the 200 is not evidence of health — it is the absence of a query.
- `/healthz` is 200 and every gated route answers 401, so the process is running, the connection pool is up and authentication reaches the database. **Only reads of catalog and service data fail.**
- The response is the canonical v1 envelope with a `requestId`, so the failure is being caught and mapped correctly — the handler is reached and throws.
- The deploy that preceded this carried `ledger-repair.sql`, which rewrites the `checksum_sha256` of 38 rows in `servana.schema_migrations` in one transaction, and the ledger it repairs includes `020-catalog-v2-expand.sql`.
- The two catalog commits in that deploy (`a73b507`, `3ac9548`) touch `providerCatalogService.ts`, `catalogAdmin.routes.ts` and `providerCatalog.routes.ts` — the provider and admin write paths. **Neither touches the customer read path**, which is what argues for schema over code.

METHOD

- Get the real error before theorising.** Pull the server log for a failing `requestId` — `envelope.ts` logs the whole exception server-side against that id while returning only `INTERNAL` to the caller. One log line likely names the missing relation or column outright. Every step below is a fallback for if that log is unavailable.
- Establish whether it is schema.** On the production database, run the catalog read query by hand. A missing table, a renamed column or a type change surfaces immediately, and distinguishes "the migration did something" from "the code did something".
- Reconcile the ledger against reality.** `ledger-repair.sql` updates checksums; it does not apply or roll back migrations. Confirm every row in `schema_migrations` corresponds to a schema state that actually exists — a repaired checksum on a migration that never fully applied is a ledger that now *lies* convincingly.
- Check `020-catalog-v2-expand.sql` specifically.** An expand migration should be additive and therefore safe. Verify that it was: if it renamed or dropped anything, or if it half-applied and the transaction did not cover the whole change, that is the fault.
- Decide roll-forward or roll-back on evidence.** Roll back to the last known-good deploy if the cause is not identified within the first thirty minutes. Restoring service is not the same as understanding the fault, and it comes first.
- Add the detector.** A synthetic check that fetches the catalog and asserts a non-empty payload, running every minute, alerting a team address. `/healthz` returning 200 through a total catalog outage is the precise gap: liveness is not readiness.
- Extend the deploy gate.** A post-deploy smoke test that requests the catalog and fails the deploy on a non-200, so the pipeline refuses to leave a broken catalog running.

ACCEPTANCE GATE — ALL MUST HOLD

- All seven listed paths return 200 with a non-empty payload, shown before and after.
- `/api/v1/search?q=aircon` returns 200 with results.
- A written cause naming the specific migration, column or query — not "a deploy issue".
- The synthetic catalog check is live and has been **watched to fail** against a deliberately broken endpoint.
- The post-deploy smoke test is in the pipeline and demonstrated blocking a bad deploy.
- A timeline: when it broke, when it was detected, how long customers were affected.

GUARDRAILS

- **Do not "fix" this by editing data until the cause is known.** A schema fault repaired by hand-editing rows leaves the schema wrong and the evidence gone.
- Any migration written to repair this is itself expand-only and transactional, per TAB 05.
- Take a database backup before any corrective DDL. The catalog is reference data; a wrong repair is worse than the outage.

DELIVERABLE

Catalog restored, a named root cause, a one-minute synthetic detector, a post-deploy smoke gate, and a written incident timeline.

TAB 02 — Reconcile the diverged backend repository

Phase	Owner	Risk	Blocks
Stop the bleeding	Backend	High	TAB 03, every future deploy

OBJECTIVE

Bring local and GitHub `main` to one history, losing nothing, so that the next deploy ships a state somebody has actually reviewed.

WHY THIS, AND WHY NOW — MEASURED

- The backend has **47 local-only** and **17 remote-only** commits. This is a genuine fork, not a stale checkout: `git push` is rejected as non-fast-forward, and `git pull` without care will produce a merge nobody has reviewed.
- Among the 47 are five verified fixes with tests, watched to fail, that have never reached production — including the fix for a defect on the **most security-relevant 401** the platform emits.
- Among the 17 are the commits deployed immediately before the catalog outage. They are in production now and are not in the local tree, so **local testing cannot reproduce the live fault**.
- Two agents and a human have been committing into one working tree. During this sweep the suite count moved between runs (6,223 → 6,226 → 6,223) and a test was red purely because a file was mid-edit.

METHOD

1. **Snapshot first.** Tag both sides — `git tag pre-reconcile-local` and `pre-reconcile-remote origin/main` — so any outcome is reversible by name rather than by reflog archaeology.
2. **Merge, do not rebase.** These commits are shared between agents and some are already deployed. Rebasing rewrites hashes that other work refers to; a merge commit records honestly that two lines of work existed.
3. **Resolve conflicts with the deployed side favoured for anything currently live**, and re-run the full suite after each conflicted file rather than at the end, so a bad resolution is attributable.
4. **Verify nothing was lost.** Assert every one of the five known-good local commits is still an ancestor of the merge result, by hash. Assert the same for the 17 remote commits.
5. **Run the full suite on the merged tree in a clean worktree**, not the shared working directory, so a concurrent edit cannot contaminate the result.
6. **Re-run the contract consumers check** — the v1 contract feeds the router, the OpenAPI document, the generated registry and its own tests, and a merge is exactly where those four fall out of agreement.

ACCEPTANCE GATE

- One history, both tags reachable, zero commits dropped — proven by hash, not by inspection.
- Full suite green on the merged tree, run from a clean worktree.
- `tests/v1-contract.test.ts` green: all four contract consumers agree.
- A written list of every conflict and how it was resolved.

GUARDRAILS

- **No force-push.** Seventeen of these commits are deployed; rewriting them detaches production from its own history.
- Do the merge when no other agent is writing to the tree. Confirm a clean `git status` that stays clean for several minutes first.

DELIVERABLE

A single reconciled history, both snapshot tags, a green clean-worktree suite, and a conflict resolution log.

TAB 03 — Ship the three verified fixes that have never deployed

Phase	Owner	Risk	Blocks
Stop the bleeding	Backend	Medium	TAB 08 rollout telemetry, TAB 10

OBJECTIVE

Deploy three fixes that are written, tested, watched to fail, and sitting unshipped — and prove on production that each landed and that nothing else moved.

WHY THIS, AND WHY NOW — MEASURED

- **Wrong successor signpost.** GET `/api/catalog`, `/api/services`, `/api/bookings` and `/api/quote` all publish `Link: </api/v1/bookings/:bookingId>; rel="successor-version"`. The catalog is not superseded by a booking read. RFC 8288 successor links are followed automatically by client generators and proxies, and **five platforms are mid-migration**. The cause was a parameter segment compiling to a pattern that matched any single segment under `/api`.
- **v1 auth failures in the legacy envelope.** Every v1 401 returns `{"status":"failed","code":"UNAUTHENTICATED"}` with no `requestId`, while every 404 and 400 returns the canonical `{"error":{"code,message,requestId}}`. A generated client gets a parser that is wrong for the single most common error class, and support cannot correlate a 401 to a log line.
- Verifying that second fix found a further defect: `verifyAuth`'s `TOKEN_REVOKED` branch writes a hybrid body carrying its own nested `error` key, so the translator's `!body.error` guard **passed the most security-relevant 401 through untranslated**. A revoked session is the one refusal a client must never mistake for a generic failure.
- **No security headers.** No HSTS, `X-Content-Type-Options`, `X-Frame-Options` or `Referrer-Policy` on an origin serving five clients. The middleware exists, mounted before the CORS delegate — which is load-bearing, because a non-whitelisted origin is still served and would otherwise carry no `nosniff`.

METHOD

1. Complete TAB 02 first. Deploying a diverged tree is how the next outage happens.
2. Capture the production before-state with `scripts/verify-deploy.sh` before.
3. Deploy through the pipeline, not by hand. TAB 01 adds a post-deploy smoke gate; use it.
4. Run `scripts/verify-deploy.sh` after. It prints a verdict per fix and **diffs status codes before and after**, failing if any moved — the additive-only rule, measured rather than asserted.
5. Confirm `TOKEN_REVOKED` and `TOKEN_EXPIRED` against a **real revoked and a real expired token**, not from source. They must arrive as distinct codes in the v1 envelope.
6. Verify HSTS deliberately before committing to a long `max-age`. It is deployed without `preload`, which is correct — `preload` is close to a one-way door and every origin on the domain inherits it.

ACCEPTANCE GATE

- `verify-deploy.sh` after exits 0: catalog successor correct, v1 401 canonical with a `requestId`, all four headers present.
- Status-code diff empty. No body, status or timing changed on any legacy route.
- A legacy 401 body byte-identical before and after — 520 legacy routes and five clients read that shape.
- Revoked and expired tokens demonstrated producing distinct codes.

GUARDRAILS

- Additive and prefix-scoped. Nothing outside `/api/v1` may change shape, status or timing.
- A 401 must not distinguish a known account from an unknown one — in status, body, timing or copy.

DELIVERABLE

Three fixes live, a passing `verify-deploy.sh` transcript, and a before/after capture proving the legacy tree did not move.

TAB 04 — A deploy that cannot silently break the catalog again

Phase	Owner	Risk	Blocks
Pipeline trust	Backend	High	TAB 16

OBJECTIVE

Make it structurally impossible for a deploy to leave production serving 500s undetected, and give every deploy a rehearsed, timed rollback.

WHY THIS, AND WHY NOW — MEASURED

- A deploy broke every catalog read and **nothing objected**. The suite on GitHub `main` is green — 278 suites, 5,957 tests — so the fault is invisible to the tests that ran.
- `/healthz` returned 200 throughout. **Liveness is not readiness**, and a health check that does not touch the product's primary read path will report health during a total product outage.
- This repository has been here before: `deploy.yml` and `release-gate.yml` once ran as independent workflows on the same trigger, the gate failed, the deploy succeeded, and a red build shipped. That specific hole was closed; this is the next one along.
- The deploy runs on a self-hosted runner on the production host and performs `npm ci`, migrations, `npm run build` and a `pm2` restart in sequence. Any step can leave the previous one half-applied.

METHOD

1. **Readiness endpoint distinct from liveness.** `/readyz` executes one representative read per critical domain — catalog, bookings, identity — and fails if any throws. `/healthz` stays as a process check.
2. **Post-deploy smoke gate.** After `pm2` restarts, poll `/readyz` and a small set of real endpoints. **Non-200 fails the deploy and triggers rollback**, rather than leaving a broken build serving traffic.
3. **Automatic rollback.** The deploy already snapshots `dist/`. Wire the smoke failure to restore the snapshot and restart, then alert. Rehearse it and record the wall-clock time to recover.
4. **Synthetic monitoring independent of the deploy** — an external check every minute against catalog, search and a booking read, alerting a team address. A deploy-time gate cannot catch a fault that appears an hour later.
5. **Alert on error rate, not only on deploy outcome.** A 500 rate above a small threshold pages someone. The current outage was found by a sweep, not by an alert.
6. **Structured error logging with the request id** surfaced where support can search it. The envelope already logs the exception against a `requestId`; make that reachable without shell access to the host.

ACCEPTANCE GATE

- `/readyz` live, and demonstrated returning non-200 while `/healthz` returns 200, against a deliberately broken dependency.
- A deliberately broken build shown to fail the smoke gate and roll back automatically, with the recovery time recorded.
- Synthetic monitoring alerting to a team-owned address, demonstrated with a real alert.
- An error-rate alert demonstrated firing.

GUARDRAILS

- The smoke gate must be read-only. It may not create a booking, a message or a ticket.
- Rollback restores code only. A migration is not rolled back automatically — see TAB 05.

DELIVERABLE

A readiness endpoint, a blocking post-deploy smoke gate, automatic rollback with a measured recovery time, and independent synthetic monitoring.

TAB 05 — Migration safety: expand–migrate–contract, enforced

Phase	Owner	Risk	Blocks
Pipeline trust	Backend	High	TAB 16

OBJECTIVE

Make a schema change incapable of taking the product down, by enforcing the expand–migrate–contract discipline in the pipeline rather than in review.

WHY THIS, AND WHY NOW — MEASURED

- The prime suspect for the outage is schema. `020-catalog-v2-expand.sql` is in the ledger, the deploy carried a ledger repair, and the failure signature is "auth works, catalog queries fail".
- `ledger-repair.sql` rewrites the checksums of 38 migrations because they were hashed from CRLF bytes on Windows and the deploy host checks out LF. The repair is correct in itself — but it shows the ledger can disagree with reality, and **a repaired checksum on a migration that never fully applied is a ledger that now lies convincingly**.
- One backend serves five clients. A contracting change — a rename, a drop, a type narrowing — breaks whichever client has not shipped yet, and the customer mobile app cannot ship quickly: it goes through two app stores.
- The deploy applies migrations before `npm run build` and the restart, so a migration that succeeds while the new code fails to build leaves a schema the running code does not expect.

METHOD

1. **Write the rule down and enforce it in CI.** Every migration is **expand-only** — add columns, add tables, add nullable, add indexes concurrently. Renames and drops are a separate, later migration, after every client that reads the old shape has shipped.
2. **A CI check that rejects contracting DDL** in a migration not explicitly marked as a contract step with a named prerequisite release. Derive the forbidden set (`DROP COLUMN`, `DROP TABLE`, `ALTER ... TYPE`, `RENAME`, `NOT NULL` without a default) rather than asking review to notice.
3. **Every migration in one transaction**, so a partial apply is impossible. Where Postgres forbids it — `CREATE INDEX CONCURRENTLY` — that migration contains nothing else and says so.
4. **Idempotent by construction.** `IF NOT EXISTS` throughout, so a re-run after a failed deploy is safe. A migration that can only be applied once is a migration that blocks recovery.
5. **Verify the ledger against the schema, not against itself.** A check that asserts each recorded migration's observable effect actually exists — the column is present, the table is present. A checksum proves the file did not change; it proves nothing about the database.
6. **Rehearse on a restored production snapshot**, not on an empty database. `fresh-db` proves migrations run from nothing; it does not prove they run against real data.
7. **Order the deploy so the schema is compatible with both the old and new code** for the duration. With expand-only migrations this is automatic, which is the point.

ACCEPTANCE GATE

- The contracting-DDL check is in CI and **shown rejecting a deliberately contracting migration**.
- Every existing migration audited for transactionality and idempotence, with exceptions named and justified.
- The ledger-versus-schema check runs against production and passes.
- A migration rehearsed against a restored production snapshot, with timing recorded.
- A written rollback procedure for a migration that cannot simply be re-run.

GUARDRAILS

- Never repair a ledger checksum without confirming the migration's effect exists in the schema.
- No migration may run without a current backup, and the restore path must have been tested this quarter.

DELIVERABLE

An enforced expand-only policy, a CI check watched to fail, a ledger-versus-schema verifier, and a rehearsed migration against real data.

TAB 06 — Unblock the push path and end the shared-tree collisions

Phase	Owner	Risk	Blocks
Pipeline trust	All	Medium	TAB 03, TAB 07

OBJECTIVE

Make it possible to ship at all, and stop two agents corrupting each other's measurements in one working tree.

WHY THIS, AND WHY NOW — MEASURED

- Both repositories reject `git push`: *refusing to allow a Personal Access Token to create or update workflow .github/workflows/... without workflow scope*. The client is blocked by `flutter-ci.yml`; the backend by `deploy.yml`, `release-gate.yml` and `fresh-db.yml`.
- Nothing this programme produced can reach production until this is resolved.** Twenty-four client commits and forty-seven backend commits are stranded.
- Two agents share one backend working tree. During this sweep a test was red purely because a file was mid-edit, the suite count moved three times, and both agents raised the same suite-count ratchet blind to each other.
- The suite-count ratchet is a shared mutable counter — a design that works for one writer and generates false failures for two.

METHOD

- Fix the credential.** Either switch both remotes to SSH — `git remote set-url origin git@github.com:...` — which is not subject to the PAT scope rule, or add `workflow` scope to the token. SSH is preferable: it removes an entire class of scope surprise.
- Verify by pushing something trivial first**, on a branch, before pushing 47 commits. A failed push of one commit costs nothing to diagnose.
- Stop sharing a working tree.** Give each agent its own `git worktree` off the same repository. They share history and objects, and cannot see each other's uncommitted edits — which is exactly the property that was missing.
- Make the suite-count ratchet resilient to concurrency.** Derive the expected count rather than pinning it, or assert a floor. A shared counter that two writers must both update is a design that guarantees a race.
- Branch protection on `main`.** Require the gate to pass before merge, so the pipeline cannot be bypassed by whoever pushes fastest.

ACCEPTANCE GATE

- A push succeeds from this machine, demonstrated.
- All stranded commits landed, and each verified present on GitHub by hash.
- Each agent operating in its own worktree; a clean `git status` in one while the other is mid-edit.
- The ratchet no longer produces a false failure when two suites land concurrently.

GUARDRAILS

- Do not strip workflow changes from commits to route around the scope rule — that silently drops CI gates, an entire release pipeline and a version pin.
- Do not weaken branch protection to land a backlog.

DELIVERABLE

A working push path, all stranded work landed, isolated worktrees per agent, and a concurrency-safe ratchet.

TAB 07 — Land the canonical v1 client layer

Phase	Owner	Risk	Blocks
Client integration	Client	Blocker	TABs 08, 09, 10, 15

OBJECTIVE

Get the canonical v1 client layer into the repository, so that the integration this whole programme exists for can begin.

WHY THIS, AND WHY NOW — MEASURED

- `grep -r "api/v1" lib/` returns **0 hits** in `Upupapp/ServanaClientAPP`. There is no `v1_endpoints.dart`, no `CanonicalAvailability`, no `api_error_mapper.dart`.
- Verified exhaustively across **every remote ref** — `main`, `dev`, `chore/16kb-alignment`, tags `build-38`, `build-39`, `build-40` — plus every pull request and fork. The layer is on none of them. This is not a matter of checking the wrong branch.
- The work exists only on a Windows developer machine, unpushed, correctly, under a standing no-push rule that has now outlived its usefulness.
- **Nine tabs of integration work cannot start.** The capability flags they switch gate a layer that is not present, and the application in customers' hands is 100 percent legacy as a direct consequence.
- The same gap blocks the viewport matrix: `test/support/screen_test_container.dart` is in the same unlanded set.

METHOD

1. **Bundle rather than push**, so nothing is exposed before it is reviewed:

```
git bundle create servana-client.bundle origin/main..HEAD
```

Transfer the file, then `git fetch ./servana-client.bundle refs/heads/main:refs/heads/incoming` and inspect before merging.

2. **Verify the bundle is complete** — `git bundle verify` — before trusting it. A truncated bundle that fetches cleanly is worse than one that fails.
3. **Merge onto the 24 commits already here**, which include the integration probe, the version gate, deep links, R8 and the analyzer triage. Neither side is a superset; both are needed.
4. **Re-run the integration probe with the client half enabled.** It currently reports that check as SKIPPED rather than green, which is the honest state. With `V1Endpoints` present it can finally diff client path constants against the backend contract — the check TAB 01 of the previous programme was written for and could never run.
5. **Reconcile the two analyzer and format baselines.** The landed work was authored on Windows; this repository now pins `eol=lf`, so expect a normalisation pass.
6. **Establish the merged baseline:** full gates from a clean clone, counts recorded, before any capability is enabled.

ACCEPTANCE GATE

- `v1_endpoints.dart`, `CanonicalAvailability` and `api_error_mapper.dart` present and compiling.
- Integration probe's client-versus-contract check **runs** and reports zero divergence — no longer SKIPPED.
- All 15 capability flags present and **all still off**. Landing the layer is not enabling it.
- Clean-clone gates green, with test and analyzer counts recorded as the new baseline.
- Nothing from either side lost, proven by hash.

GUARDRAILS

- **Do not enable a capability in this tab.** Landing and enabling are separate decisions with separate evidence.
- Do not infer client state from the backend contract's `callers` field — that is a record the backend keeps, not an observation of the client.

DELIVERABLE

The canonical layer landed and merged, the probe's client half running, all flags off, and a recorded merged baseline.

TAB 08 — The migration harness: evidence gate, rollout, rollback

Phase	Owner	Risk	Blocks
Client integration	Client + Backend	High	TABs 09, 10

OBJECTIVE

Build the machinery that lets one capability move from legacy to canonical with evidence before, observation during and a one-step reversal after — so the waves are repetitions of a rehearsed procedure rather than improvisations.

WHY THIS, AND WHY NOW — MEASURED

- The canonical layer sits behind 15 capability flags, all off, all build-time. That gate was correct when v1 was undeployed. All 105 implemented v1 endpoints now answer, so the gate is no longer protecting against a missing backend.
- The flags are deliberately **not remote-configurable** — a flag a server can flip is a flag an attacker can flip. That decision stays. Its consequence is that a bad wave is reversed by shipping a build, which is why each wave needs a rehearsed rollback and a bounded blast radius.
- A prior remediation found `CustomerBooking` was lossier than the screen it replaced: a blind swap would have regressed `totalAmount` to a zero-peso display, plus `serviceOptionName`, `branchName`, the three-source status reconciliation, `latitude`, `longitude`, `downPayment` and `numberOfPersonnel`. **This wave must re-run that proof, not assume it.**
- Today's outage is the argument for the observability half: a transport that starts failing must be visible as a transport failure, not as an app that quietly shows nothing.

METHOD

1. **Write the evidence checklist** a capability must satisfy before it ships: every path answers non-404 with the contract method; both data sources satisfy one interface; the **domain object** from each is equal for a real account; every error code the endpoints emit is mapped; anything the endpoint set cannot serve is reported unsupported rather than silently degraded.
2. **Build the differential harness.** For one canary account, call both transports for every read and assert deep equality of the **domain object, not the wire payload**. The wire shapes differ by design; the domain objects must not.
3. **Fail by name on a dropped field.** A canonical read that adds a field is fine; one that drops a field legacy supplied is a regression, and the seven named above must each fail individually rather than as an opaque inequality.
4. **Transport observability.** Every repository records which source answered; a diagnostics screen shows per-capability transport, request count, p50/p95 and error rate. Instrument through the existing analytics privacy filter — transport, capability, endpoint id, status and latency are safe; nothing resembling an identifier or a location may join them.
5. **Provision a canary account on production** with real booking history, and record how to reset it.
6. **Rehearse the rollback end to end** and record the wall-clock time: revert the define, rebuild, distribute, confirm the diagnostics screen reports fully-legacy. **If it exceeds one hour, TAB 14 becomes a prerequisite rather than a parallel workstream.**
7. **Define wave exit criteria** used by every later wave: 48 hours on the canary with zero canonical-transport errors the legacy transport did not also produce, and p95 no worse than legacy by more than 20 percent.

ACCEPTANCE GATE

- Differential harness green for at least one capability, and **demonstrated failing when a field is deliberately dropped** from a canonical mapper.
- Diagnostics screen correct with flags on and off.
- Rollback rehearsed and timed, the number recorded.
- The checklist committed and referenced by every later wave.

GUARDRAILS

- The master switch and capability list stay build-time. No network input may enable canonical traffic.
- An unrecognised capability name fails toward legacy rather than throwing at startup.
- The canary is a real account on production. No test bookings a provider could be dispatched to without warning.

DELIVERABLE

An evidence gate, a differential harness watched to fail, transport diagnostics, a provisioned canary, and a timed rollback.

TAB 09 — Wave 1–2: catalog, search, identity and customer profile

Phase	Owner	Risk	Blocks
Client integration	Client	Medium	TAB 10

OBJECTIVE

Move the read-only public surface and the identity surface onto `/api/v1`, and close the password-reset gap that a false comment has been holding open.

WHY THIS, AND WHY NOW — MEASURED

- All ten canonical catalog endpoints and `/api/v1/search` answer **200 unauthenticated** when healthy — the only part of the surface verifiable end to end without a session, which makes it the correct first wave. **Subject to TAB 01: they are 500 today.**
- The canonical catalog emits qualified references such as `service:180`, resolving the four-way identifier collision the legacy catalog leaves to the caller. Server-side search replaces `String.contains` over a downloaded catalog with backend term expansion — which is what makes `aircon` and `air conditioning` resolve to the same services.
- The catalog response carries `Cache-Control: public, max-age=300` and a weak `ETag`. The client cache should honour them rather than reimplementing a private expiry policy.
- Identity paths are mounted and gated: `/api/v1/me`, `/me/settings`, `/me/security`, `/me/completion`, `/me/notification-preferences`, `/customer/profile` and the full address set all return 401 rather than 404.
- **Password reset does not exist in the app.** "Forgot password?" raises a "coming soon" snackbar, and the comment justifying it claims reset goes through Firebase — but `sendPasswordResetEmail`, `confirmPasswordReset` and `verifyPasswordResetCode` have **zero hits** in `lib/` and `test/`. The comment is false. `POST /api/v1/auth/forgot-password` answers 200. **A customer who cannot reset a password is a customer who is gone.**

METHOD

1. Run the TAB 08 checklist for `catalog`, `search`, `identity` and `customerProfile`.
2. Differential-harness the full catalog tree and search, including a service with add-ons and one that is unavailable — availability and add-on identity are where the transports have most room to disagree.
3. **Re-verify the category-key trap and pin it.** Home grid keys use underscores (`beauty_wellness`, `hair_nails`); the backend slug generator produces hyphens. A campaign registered against a key the catalog does not use returns null, so there is no popup, no exception, **and the tests still pass**. Route by `catalog_categories.id`, never by slug.
4. Wire the cache to the transport: send `If-None-Match` and treat 304 as a hit. Confirm against production that a conditional request actually returns 304 before relying on it.
5. **Delete the false comment** in the identity compatibility source and replace it with what is true. A comment that misdescribes the system is worse than none, because it stops the next reader checking.
6. Implement password reset on both transports, so it works with the capability off as well as on. **Handle the anti-enumeration contract deliberately** — the endpoint returns 200 for an empty body, which is correct; the screen must show the same confirmation regardless and never branch on the response in a way a user can observe.
7. Reset completion requires the deep links in TAB 13. Until that lands, ship the request half and say plainly in the report that completion runs through the web fallback.
8. Confirm logout clears everything: composition cache, session store, secure storage and the operation journal, on both transports.

ACCEPTANCE GATE

- Differential harness green across `catalog`, `search`, `/me`, `/customer/profile` and the address list, including a non-default primary address.
- Category-key assertion committed and **shown failing when a slug is substituted for an id**.
- A 304 demonstrated from production against a stored `ETag`.
- Password reset demonstrated end to end on both transports, with identical behaviour for a registered and an unregistered address.
- **Sign-in and registration confirmed still on the compatibility path, by test** — enabling `identity` must not move them.
- Logout leaves no cached customer data, demonstrated by inspecting storage.

GUARDRAILS

- No screen may learn which transport answered. If a widget needs to know, the abstraction is in the wrong place.
- No reset response may distinguish a known address from an unknown one, in status, body, timing or copy.
- Offline: with the capability on and the network down, the catalog cache must still serve and search must degrade to a stated offline state rather than an error blaming the customer's connection.

DELIVERABLE

Four capabilities enabled in a canary build, a working password reset, the category-key pin, and a corrected comment where a false one stood.

TAB 10 — Waves 3–6: notifications, bookings, lifecycle and money

Phase	Owner	Risk	Blocks
Client integration	Client	High	TAB 16

OBJECTIVE

Complete the migration through the state-changing and money surfaces, adding three customer-facing capabilities the app has never had, and state explicitly what stays legacy forever.

WHY THIS, AND WHY NOW — MEASURED

- **Three live endpoints nobody calls.** The client asserts in three files that legacy notification dismiss has no canonical successor. `DELETE /api/v1/notifications/:key` is implemented and answers 401. Two conversation endpoints — attachments and message report — are live and unnamed by the client. **The app renders message attachments it has no path to send.**
- **The booking-read trap.** Re-run the losslessness proof for the seven fields named in TAB 08. A related parsing hazard must stay pinned: `(x as num?)` throws on a String rather than yielding null, so an amount chain falling back to `double.tryParse` could never reach its own fallback. Postgres numeric arrives as integer, double or string depending on value and driver.
- **Booking creation stays legacy permanently.** `POST /api/bookings` is classified KEEP with no canonical successor and none planned. Documented, not deferred.
- **Tracking changes a privacy boundary, not a URL.** The legacy pair returns a provider position in every state; the canonical route withholds it unless the booking is trackable, a provider is assigned and the movement window is open, answering 200 with a `visibility.reason`. Enabling it is the client giving up its own idea of when a location may be shown — the correct direction.
- **Payments and disputes add capability that does not exist.** There has never been a legacy payment-state endpoint or a customer refund route, and no customer dispute route — `canOpenDispute` is false everywhere precisely because the compatibility transport cannot serve one.
- **31 of 36 v1 error codes are unmapped.** Five booking refusal codes share a status and collapse into one generic message. `RATE_LIMITED` deserves a wait-and-retry; `TOKEN_EXPIRED` a silent re-auth; `TOKEN_REVOKED` a sign-out and an explanation.

METHOD

1. **Map the full error vocabulary first.** Generate the code list from the backend rather than transcribing it, decide for each a failure kind, a customer sentence and whether the app can act on the customer's behalf. Codes with an automatic recovery should never reach the customer as an error. Add a completeness test that fails when the backend adds a code, and mutation-test it.
2. **Reclaim the three unused endpoints** and add a regression test that reads the contract and fails if any call pinned to the compatibility transport has acquired an implemented canonical successor. The comment deletions are instances; this is the general fix.
3. **Booking reads:** field-level equality for a booking in every reachable state, naming the seven fields individually. Include a change order, a down payment, multiple personnel and a branch, so each field is exercised rather than nominally covered. Verify the money parser against integer, double and string amounts from **real production rows, not fixtures** — a fixture agrees with whatever its author expected.
4. **Lifecycle:** exercise idempotency for real — submit a cancel, kill the process mid-flight, relaunch, let the journal resume. One cancellation, not two and not zero. Exercise reuse, conflict, terminal and not-cancellable paths, each producing distinct, truthful copy.
5. **OTP:** request, wrong code, cooldown, resend, correct code — with a network capture proving **no code appears in any query string**. A query string is written to the access log on every request, and the OTP is what proves the customer is present.
6. **Tracking:** render `visibility.reason` rather than a blank map, and make a legacy fallback **structurally impossible** — a fallback when the canonical route withholds would defeat the entire privacy boundary.
7. **Money surfaces:** gate the payment screen on `hasPaymentDetail` so a zeroed breakdown is never rendered as a price; gate refund on `canOfferRefund`; state the timeline in confirmation copy and confirm it matches what operations actually

does.

8. Write `REMAINS_LEGACY.md` : booking creation, sign-in, registration, review read/edit/delete/list/report — each traced to a contract disposition.

ACCEPTANCE GATE

- All 36 v1 codes mapped, the count asserted against the contract, the completeness test **shown failing when a mapping is removed**.
- Field-level equality demonstrated for every booking state, the proof shown failing when a field is dropped.
- Idempotency demonstrated across a process kill, with the resulting booking state shown.
- OTP ceremony demonstrated with a network capture proving no code in a query string.
- Tracking withheld for a non-trackable booking with the reason rendered, and no legacy fallback reachable.
- A refund and a dispute each demonstrated end to end, with the record visible to operations.

GUARDRAILS

- **No ghost success.** Every mutation reflects the server outcome; an unknown outcome is resolved by re-reading, never assumed. A ghost success on a cancellation is the worst failure in this wave.
- The booking state machine is owned by the backend. The client proposes transitions and never computes them.
- Retries carry a stable idempotency key by construction, or they are duplicate booking actions.
- **Money is the highest-risk surface.** No client-side arithmetic may produce a figure a customer reads as authoritative — the server total is the total. Never display an amount from a field that is always null.

DELIVERABLE

All fifteen capabilities enabled, a complete error vocabulary, three new customer-facing surfaces, and a written statement of what remains on legacy and why.

TAB 11 — Close the 30 undeclared legacy routes

Phase	Owner	Risk	Blocks
Client integration	Backend	Medium	TAB 16

OBJECTIVE

Give every legacy route the customer app actually calls a disposition, so the deprecation clock can see all of them.

WHY THIS, AND WHY NOW — MEASURED

- The shipped client constructs **70** legacy routes. **30** have no entry in `V1_CONTRACT[].legacy` — no disposition, no successor, no place on the deprecation clock.
- They are not obscure. They include `/api/quote`, four `/api/services` routes, five `/api/reviews` routes and the entire ten-route `/api/support` surface.
- **94 routes are already classified `ALIAS_TEMPORARILY` and production emits `Deprecation: true` on them.** When those retire, these 30 are the ones nobody counted.
- This was invisible to a contract-derived probe and only appeared when routes were enumerated from the **client's own call sites**. Probing either list alone leaves a hole exactly where field breakage would appear.
- The itemised list already exists at `docs/integration/M6_UNDECLARED_LEGACY_ROUTES.md`, grouped by domain and regenerable from the probe.

METHOD

1. Work the generated list domain by domain. For each route decide `KEEP`, `ALIAS_TEMPORARILY`, `CANONICALIZE`, `ROLE_SPECIFIC` or `RETIRE` — and for anything but `RETIRE`, record why it is not simply deleted.
2. **Add the entries to the contract array**, which is the one source the router, the OpenAPI document, the generated registry and the deprecation matcher all derive from. Adding it in one place gives it a successor link and a place in the migration matrix automatically.
3. **Extend the probe into a gate.** It already computes this set; make a non-empty set fail CI, so a newly added client call cannot escape classification.
4. Compare shapes, not literal text — the contract writes `:bookingId` and a call site yields a concrete id. Comparing literally inflated this figure from 30 to 50 once already.
5. Re-run the deprecation self-consistency test afterwards: every notice must still match its own path, or a route's headers silently vanish.

ACCEPTANCE GATE

- Zero routes the client constructs are undeclared — asserted by the probe, not by inspection.
- The CI gate **shown failing** when a client call with no contract entry is added.
- Every new entry carries a disposition and a note; every `ALIAS_TEMPORARILY` names an implemented successor.
- All contract consumers still agree; every deprecation notice still matches its own path.

GUARDRAILS

- Classification is a contract decision, not a client one. Do not silence the gate by removing a client call that customers depend on.
- Adding an entry changes published headers. Additive only — no status, body or timing may move.

DELIVERABLE

Thirty routes classified, the contract updated, and a CI gate that prevents the next unclassified call.

TAB 12 — Finish the Android and iOS release paths

Phase	Owner	Risk	Blocks
Release engineering	Client	High	TAB 14, TAB 16

OBJECTIVE

Produce a release artefact that is verifiably shrunk, obfuscated and correctly signed on Android, and a CI-produced TestFlight build on iOS — then prove both on a device.

WHY THIS, AND WHY NOW — MEASURED

- R8 is now enabled and verified on a real artefact: DEX **26.74 MB** → **7.07 MB (~73.6%)**, **68.2%** of classes renamed. But the bundle was signed with a **throwaway key** generated for the measurement and deleted. **Nothing is established about the real upload key.**
- The release build was **unbuildable** until this programme fixed it: Gradle 8.13 below Flutter's minimum 8.14.0, and Kotlin 2.0.0 below 2.2.20 — **with no repository change**. CI resolved an unpinned `channel: stable` and the floors moved underneath it. Now pinned to 3.47.0.
- A release build with no keystore previously produced an **unsigned artefact** and proceeded; the comment claiming a debug fallback was wrong for AGP 8.x. A fail-fast guard now refuses it — verified, and verified not to affect debug builds.
- iOS had `build-ios` and no `release-ios`: every TestFlight build was a manual act on a laptop, unversioned and unattributable. A pipeline now exists but has **never run**.
- `DEVELOPMENT_TEAM` is `2K2SF7NRQP` for Runner and `CAB884NRSN` for RunnerTests — the kind of mismatch that costs a day at signing time.
- Local Android builds fail on a default machine: Android Studio bundles Java 25 and Gradle 8.14.3 accepts up to 24. CI pins Java 17 and is unaffected.

METHOD

1. **Verify the artefact, not the configuration.** Run `apksigner verify --print-certs` on the real bundle and record the fingerprint. Confirm it is the expected Play upload key — mistakes here are close to unrecoverable, and this project has been through Play re-signing requiring both certificates.
2. **Upload the R8 mapping to Crashlytics as part of the release job**, and verify a deliberately triggered release-mode crash symbolicates. An obfuscated build without a mapping upload trades readable crash reports for a smaller binary — a bad trade made silently.
3. **Run the full functional pass against the R8 artefact, on a device.** R8 failures appear only in the shrunk build, at the moment a reflective lookup runs. A green build is not evidence the app works. Add keep rules only for what is **observed** to break — a rule added "just in case" defeats the shrinker it was meant to accommodate.
4. **Audit the Apple portal prerequisites before the first release run:** Sign in with Apple, push, Associated Domains, and the APNs key in Firebase for `servana-59bee`. A pipeline that builds an archive the portal will reject wastes the whole cycle.
5. Reconcile the `DEVELOPMENT_TEAM` mismatch.
6. **Run `release-ios` and install from TestFlight on a real device.** Confirm push arrives, Sign in with Apple completes, and a Universal Link opens. Re-run the job from a clean state to prove it does not depend on runner or keychain residue.
7. Record the per-ABI download size, which is what determines install conversion on the mid-range Android hardware this market runs.

ACCEPTANCE GATE

- Certificate fingerprint recorded and matched to the expected upload key.
- A release-mode crash symbolicated in Crashlytics from the uploaded mapping.
- Full functional pass executed against the R8 artefact on a device.
- A TestFlight build produced **entirely by CI** and installed on a real device.
- Portal prerequisites confirmed with evidence rather than assumed.

GUARDRAILS

- Never commit `key.properties`, a keystore, a mapping file, a certificate or an API key. The ignore rules were extended and verified with `git check-ignore`; confirm before the first run.
- Do not change the signing key. The upload key in use is the one Play expects.

DELIVERABLE

A verified-signed, obfuscated Android bundle with symbolicated crashes, a CI-produced TestFlight build installed on a device, and a confirmed portal prerequisite list.

TAB 13 — Deep links: host the association files and verify on device

Phase	Owner	Risk	Blocks
Release engineering	Client + Backend	Medium	TAB 09 completion, TAB 16

OBJECTIVE

Give the app a verified external entry point on both platforms, so email links, push notifications and shared URLs open the right screen.

WHY THIS, AND WHY NOW — MEASURED

- The client half is built: a pure resolver, a coordinator handling cold and warm start, `autoVerify` intent filters, an `associated-domains` entitlement, and both association files authored and valid. **None of it is hosted.**
- **This blocks password reset from completing in-app**, which is the largest funnel gap in the application.
- The failure direction is currently safe: verification fails while `assetlinks.json` is unhosted, so links open in the browser. **The moment the files are hosted, the client half must be present and correct** — otherwise every claimed URL opens the app to nothing.
- A prior draft of exactly that shipped: the resolver existed with **zero callers**, and would have sprung on whoever hosted the files. It now has a caller, wired in `main.dart`.

METHOD

1. Serve `/.well-known/assetlinks.json` and `/.well-known/apple-app-site-association` over HTTPS on all three hosts. **The Apple file as `application/json`, no extension, no redirect** — that single detail is the most common cause of a silent Universal Link failure.
2. **Confirm the well-known path is not shadowed** by an existing route before deploying. A wildcard route has eaten a new sibling on this backend before.
3. Fill **both** Android fingerprints: the Play app-signing certificate, which is what devices actually see, and the upload key, so internal-testing builds verify too. **Shipping only one is the usual mistake.**
4. Enable Associated Domains on App ID `com.servana.client` in the Apple portal.
5. **Verify through the platform, not by observing a tap.** `adb shell pm get-app-links com.servana.serviceclient` must report *verified*. On iOS, open from Mail and Notes — Safari's address bar deliberately does not trigger Universal Links, and testing there is the classic false negative.
6. Verify cold start and warm start separately. A link that launches a terminated app arrives once at startup; handling only the stream loses the majority of real links.
7. Verify the signed-out and forbidden cases: a held destination that resumes after sign-in, and a booking belonging to another account producing a clear refusal rather than a partially rendered screen.

ACCEPTANCE GATE

- Android verification confirmed **verified** on a real device via the platform tool, not inferred.
- iOS Universal Links opening from Mail and Notes, cold and warm.
- Every route in the table demonstrated, including signed-out and forbidden.
- Password reset completing in-app end to end, closing the TAB 09 remainder.

GUARDRAILS

- A deep link is untrusted input. It must not trigger a mutation, and must not carry a token or one-time code in the path or query. The reset link's token is deliberately dropped; the screen asks for the code.
- The host allow-list is an exact set, never a suffix match — otherwise `servana.com.ph.evil.com` is a valid link.

DELIVERABLE

Hosted association files, verified App Links and Universal Links on real devices, and an in-app password reset completion.

TAB 14 — Publish and rehearse the version gate

Phase	Owner	Risk	Blocks
Release engineering	Client	High	TAB 08 rollback budget, TAB 16

OBJECTIVE

Make a shipped build retirable on purpose, and measure how long that takes — because TAB 08's rollback plan depends on the number.

WHY THIS, AND WHY NOW — MEASURED

- The gate is built: pure policy, a Remote Config repository with last-known-good caching, a frequency-capped soft prompt, a hard block, and a barrier mounted above the router so a blocked build cannot make an authenticated request. **Not one parameter is published.**
- Before this, `update_repo.dart` wrapped Play's update flow and had **zero callers**; nothing enforced a minimum version, and `in_app_update` is Android-only, so even called it covered half the fleet.
- **94 legacy routes are ALIAS_TEMPORARILY with Deprecation: true live.** When they retire, every unmigrated installed build breaks, and without this gate there is no way to tell those builds to upgrade or to count them.
- The capability flags are build-time by design, so **the only way to retire a bad wave fleet-wide is to require an upgrade.** That makes the propagation delay a hard input to TAB 08: if reverting takes over an hour, this tab is a prerequisite rather than a parallel workstream.

METHOD

1. Publish the six `version_gate_*` parameters in `servana-59bee`. An unconfigured project behaves exactly as if the gate were absent, because the in-app default minimum is 0.
2. **Confirm the build you intend to require is live on both stores** and has had time to propagate. Blocking on a build customers cannot install is a total outage.
3. **Rehearse on real Android and real iOS devices:** publish a minimum above the installed build, confirm the block appears with its explanation and store link, restore, confirm it clears. **Record the propagation delay** and hand it to TAB 08.
4. Verify the failure policy behaves as designed: never-fetched allows; a previously-fetched minimum is **enforced offline**; an unreadable build number allows; a newer schema version is ignored entirely rather than half-applied.
5. Confirm the soft prompt is capped and the hard block is never suppressed by that cap.
6. Emit build number and canonical capability set as analytics user properties through the existing privacy filter, so fleet distribution and migration progress are both measurable.
7. Name an owner for publishing a minimum version. It is the single most dangerous parameter in the project — it can brick every installed app — and should be treated like a deploy.

ACCEPTANCE GATE

- Hard block and soft prompt each demonstrated on a real Android and a real iOS device.
- Fetch failure demonstrated failing open; a cached minimum demonstrated still enforcing offline.
- Build-number distribution visible in analytics within a day of the canary.
- **Propagation delay measured and recorded**, and TAB 08's rollback plan updated against it.
- A named owner for the minimum-version parameter.

GUARDRAILS

- The gate may block the app; it may never silently change behaviour. No feature may be enabled or disabled through this channel.
- The blocking screen must state why and offer the store. A dead end with no explanation generates one-star reviews that outlive the incident.
- Remote toggling of the canonical capability flags stays out of scope, permanently.

DELIVERABLE

Published parameters, a rehearsal on both platforms, a recorded propagation delay, fleet telemetry, and a named owner.

TAB 15 — Front-end integrity: coverage matrix and dead code

Phase	Owner	Risk	Blocks
Quality	Client	Medium	TAB 16

OBJECTIVE

Cover every screen at every supported viewport and text scale, and remove the unreachable code that hides a crash behind a plausible-looking flow.

WHY THIS, AND WHY NOW — MEASURED

- A STITCH sweep found **6 unreachable screens, 1,417 dead lines**, computed transitively from the router so a cluster reachable only from another orphan still counts as dead.
- The worst is a complete checkout branch inside a live module. `CheckoutJobOrderScreen` is unrouted, and its "Continue" button calls `goNamed` on **two route names the router does not declare** — for both cash and QR. `goNamed` on an unregistered name throws. **No test touches any of the three.** It is not live today; wire it in and it crashes immediately, on the money path.
- The viewport matrix covers **13 of 62** screens — and those 13 already found a heading clipped at every viewport and every text scale on the **safety** screen, and the rewards screen clipping by 411 pixels at 320×568 at scale 2.0. Both were found in about a minute by a throwaway probe, because nothing else in the suite builds a screen at a viewport.
- The 49 uncovered screens include the booking flow, checkout, payment and the address form — where a clipped control is not cosmetic but a customer who cannot complete a purchase.
- The DI graph is intact: 73 registered, 49 resolved, zero missing. Route reachability is 55 of 56, the exception deliberate.

METHOD

1. **Delete the three orphaned `job_order` screens.** 712 lines, zero references, zero coverage, and a latent crash if revived. Deleting proves the point — if the compiler stays quiet, nothing used them. Decide the other three orphans deliberately; `ServiceCategoryListScreen` at 495 lines may be catalog-v2 scaffolding.
2. **Enumerate the screens mechanically and assert the count in a test**, so a new screen that skips the matrix fails the build rather than joining a backlog.
3. Extend the test container to cover what is currently excluded — the Home and booking flows need their live store graph, the permissions screen needs Firebase Messaging faked at the boundary. Both are solvable at the injector, and both are why coverage stopped at 13.
4. Run every screen at each supported width and from text scale 1.0 to the declared maximum of 2.0. **Assert no overflow in a way that fails the test** — release builds clip silently rather than throwing, which is exactly why this is invisible in production.
5. Add semantics assertions: every interactive element has a label, every tap target meets the platform minimum, focus order follows reading order. Check contrast on **real rendered colours**, not on the palette definition — a token that passes in isolation can fail over the surface it is drawn on.
6. **Add a route-integrity test** that fails when any `goNamed` / `pushNamed` target is not declared by the router. That is the general fix; deleting the orphans is the instance.
7. Carry forward the known harness trap: the API client wraps every request in a 30-second timeout, so a screen that loads on build arms a timer and the binding fails at teardown. Tear down and pump past the window before asserting; a timer still pending after that is a real leak.

ACCEPTANCE GATE

- All screens covered, with the count asserted from the source tree.
- Zero overflow across the full viewport and text-scale matrix.
- The harness **demonstrated failing** on a deliberately introduced overflow.
- The route-integrity test **demonstrated failing** on a navigation to an undeclared route.
- Zero unreachable screens, or each survivor justified in writing.
- No pending-timer failures at teardown, with any real leak fixed rather than suppressed.

GUARDRAILS

- **Do not raise the supported text scale to make a failure disappear.** The declared maximum is a commitment.
- A test can only find what it renders. A screen registered but not built is not covered, whatever the count says.
- New Phase C surfaces — payment detail, refunds, disputes, attachments, password reset — enter the matrix as they are built, not afterwards.

DELIVERABLE

Full-matrix coverage with an asserted screen count, a route-integrity gate, the dead checkout branch removed, and a harness proven to fail.

TAB 16 — Launch readiness review and go / no-go

Phase	Owner	Risk	Blocks
Launch	All	High	nothing downstream

OBJECTIVE

Establish from measurement — not from this document — that the application and its integration are ready for a public release, and record honestly whatever is not.

WHY THIS, AND WHY NOW — MEASURED

- Fifteen tabs produce evidence; this one decides. The decision must rest on the state of the system on the day it is taken. **This document opened with a live outage that no alert caught** — that is the standard of scepticism required.
- Several launch dependencies sit outside every repository: the Play listing, the Apple portal capabilities, the APNs key in the Firebase console, and the association files. Each needs an owner and a confirmation.
- The Play listing was previously rejected for the sibling worker application because **the reviewer was given a customer account**. This is the customer application; say so plainly and provide an account that shows what the reviewer is asked to review.
- The platform has 109 bookings and zero completions ever recorded.** A launch plan should be honest that this is a first real production load, and instrument accordingly rather than assume the shipped paths have been exercised.

METHOD

- Re-run the integration probe against production on the release-candidate day** and attach it. Do not cite this document as evidence of the backend state.
- Re-run all gates and the viewport matrix against the **exact artefacts being submitted**, verified against their build commit — not a rebuild that happens to share a version number.
- Complete the Play data-safety declaration and the Apple privacy nutrition labels from what the app actually collects. Reconcile both against `PrivacyInfo.xcprivacy` and against the analytics allow-list, which is the only authoritative statement of what leaves the device.
- Confirm every out-of-repository prerequisite with evidence: Play signing and listing, Apple portal capabilities, the APNs key for `servana-59bee`, and the association files being served correctly.
- Verify monitoring end to end.** Trigger a release-mode crash and confirm it symbolicates; confirm the TAB 04 readiness endpoint and synthetic checks alert; confirm transport diagnostics report correctly on the release build; confirm RASP alerts reach the team alias rather than an individual's mailbox.
- Rehearse rollback once more on the release candidate — the version gate from TAB 14 and the capability revert from TAB 08 — both timed.
- Write the support runbook for the surfaces this launch adds.** Refunds, disputes, message reports and password reset all create obligations somebody must answer; a customer-facing promise with no operational owner is worse than the missing feature was.
- Hold the go/no-go. Record the decision, the evidence, named owners for anything outstanding, and the date it will be revisited.

ACCEPTANCE GATE

- Probe re-run attached and dated, with zero implemented v1 paths and zero client legacy paths returning 404 or 500.
- Gates and matrix green on the submitted artefacts, verified against their build commit.
- Store declarations complete and reconciled against the analytics allow-list.
- Every out-of-repository prerequisite confirmed with evidence rather than assertion.
- Crash symbolication, readiness monitoring, transport diagnostics and RASP alerting each demonstrated on the release build.
- Rollback rehearsed and timed on the release candidate.
- A written decision with named owners.

GUARDRAILS

- **Do not treat this document as evidence.** Every claim in it was measured on 19 August 2026 and must be re-measured on the day the decision is taken.
- A no-go on one item is not a no-go on the release unless that item is genuinely blocking. Say which, and why.
- **Report what is not done as plainly as what is.** A launch review that reports only green has not been run.

DELIVERABLE

A dated readiness report with re-measured evidence, complete store declarations, verified external prerequisites, a rehearsed rollback, a support runbook, and a written go or no-go.

How to run this master command

Each tab is a complete command. Give one tab at a time. A tab is finished when every line of its acceptance gate is satisfied **with evidence attached** — not when the code is written.

RULES THAT APPLY TO EVERY TAB

- **Re-measure before acting.** Every number here was true on 19 August 2026 and some will not be when a tab is run. This document exists because a recorded position was wrong twice: once about a backend believed undeployed, and once about a catalog believed healthy.
- **A green gate proves nothing until it has been watched to fail.** Every detector in this document carries a mutation step, and the mutation must be shown to land. During this sweep, four defects were found in the sweep's own tooling that way.
- **Prefer the running system to the source.** A live probe found an outage that a green 5,957-test suite did not.
- **Do not let a fake agree with you by construction.** A test double implementing the client's own interface matches the client's field names whatever the server accepts — assert at the HTTP seam.
- **Report what is not done as plainly as what is.** A tab that closes eight of nine items and says so is more useful than one that reports nine and means eight.

THE SHORTEST PATH TO A LAUNCH

Order	Tabs	Why these first
1	01	The product is down. Nothing else is a priority while a customer cannot browse or book.
2	02, 06	One history and a working push path. Without these nothing ships, and pushing a diverged tree is how the next outage happens.
3	03, 04, 05	Ship the three verified fixes, then make the pipeline incapable of repeating TAB 01 silently.
4	07	Land the canonical layer. Nine tabs are blocked behind it, and the shipped app stays 100% legacy until it lands.
5	08, 09	The harness, then the two lowest-risk waves. TAB 09 also closes the password-reset funnel gap.
6	12, 14	A real signed artefact and the ability to retire one. TAB 14's propagation number feeds TAB 08's rollback budget.
7	10, 11, 13, 15	The remaining waves in order of blast radius, the undeclared routes, deep links and the coverage matrix.
8	16	The decision.

ONE THING TO CARRY OUT OF THIS SWEEP

A deploy took every catalog read to 500 and **nothing objected**. The test suite was green — 278 suites, 5,957 tests. `/healthz` returned 200 throughout. The fault was found by a person running a probe, not by a system watching.

Three findings in this document share that shape: a claim that was accurate when written, became false when something else moved, and had no detector in between. TABs 01, 04 and 11 exist to make that class of staleness impossible to repeat — and TAB 16 exists to refuse to trust this document on the day it matters.